

MEEN 612 – Final Project 2: Python Control

Name: Naveen Batakurki

UIN: 537001208

1. Introduction:

In this final project for MEEN 408/612, we design and implement a Cartesian impedance controller for a UR5e robotic manipulator using Drake. The controller commands joint torques to track a smooth rectangular trajectory in Cartesian space while exhibiting desired stiffness and damping behaviour. Task-space forces are mapped to joint torques via the Jacobian transpose using a dynamically consistent inertia matrix, with a null space controller maintaining a comfortable joint posture. We demonstrate accurate trajectory tracking and axis-decoupled responses to external bias forces, connecting course concepts such as Jacobian-based force mapping, reflected inertia, and feedback linearization to a practical robot control implementation.

2. Background & Theory:

In my project I have implemented a cartesian impedance controller, which regulates the end effector interaction with its environment by shaping its effective stiffness and damping in the task space rather than joint space.

2.1 Jacobian and task-space mapping: The Jacobian relation joint velocities to end effector velocity ($\dot{x} = J\dot{q}$) and its Jacobian transpose does the reverse. It converts a cartesian force into joint relationship to reach the motors.

2.3 Task-space inertia: To make the controller behave consistently regardless of the robot configuration, I am computing the task space inertia matrix and multiplying with desired acceleration by Lambda converts it into the correct force need for the actually motion.

2.4 Control law: the desired end effector acceleration is given by this formula:

$$\ddot{x}_{cmd} = \ddot{x}_{des} + K_p \cdot e + K_d \cdot \dot{e} - \dot{J}\dot{q}$$

where e is the position error, $\dot{e}$ is the velocity error, K_p and K_d are diagonal stiffness and damping gain, and $J\dot{q}$ correct for the changing Jacobian.

The final joint torque is:

$$\tau = J^T \Lambda \ddot{x}_{cmd} + Cq - \tau_g$$

2.5 Axis decoupling: Because K_p and K_d are diagonal matrices, each axis is controlled independently. A force applied only in X causes a response only in X and Y and Z remain constant. This is decoupling property tested in the bias force experiment.

2.6 Critical damping: Gains were selected to satisfy critical damping ($\zeta = 1$)

3. Controller Design:

3.1 Control architecture: At every timestep the controller reads the robot's current state from Drake and computes the dynamics needed for the control law.

```
# ---- Dynamics ----
Mq = np.asarray(self.plant_.CalcMassMatrix(self.plant_context),
Cq = np.asarray(self.plant_.CalcBiasTerm(self.plant_context),
tau_g = np.asarray(self.plant_.CalcGravityGeneralizedForces(self.plant_context),

Jdot_v = np.asarray(
    self.plant_.CalcBiasTranslationalAcceleration(
        self.plant_context,
        JacobianWrtVariable.kV,
        self.tool_frame,
        np.zeros(3),
        self.world_frame,
        self.world_frame
    ),
    dtype=float
).reshape(3,)
```

3.2 Cartesian error and force computation. Position and velocity errors are computed in Cartesian space and converted to a task-space force through the inertia matrix Λ :

```
ramp = min(time_now / self.ramp_time, 1.0)
e_x = ramp * (x_des - x_now)
e_xd = ramp * (xd_des - self.xdot_filt)
xdd_des_ramped = ramp * xdd_des
```

```
xdd_cmd = xdd_des_ramped + self.Kp_cart @ e_x + self.Kd_cart @ e_xd - Jdot_v
```

```
F_task = Lambda @ xdd_cmd
F_task = np.clip(F_task, -self.F_max, self.F_max)
```

3.3 Joint torque command. The Cartesian force is mapped to joint torques via J^T , combined with gravity and Coriolis compensation, and clipped to hardware limits:

```
tau_task = J.T @ F_task
```

```
tau_control = tau_task + tau_null + Cq - tau_g
tau_control = np.clip(tau_control, -self.tau_limit, self.tau_limit)
```

3.4 Velocity filtering. Raw Cartesian velocities are smoothed with an exponential filter ($\alpha = 0.15$) to reduce torque chatter from noisy velocity estimates:

```
xdot_raw = J @ qd
self.xdot_filt = self.alpha * xdot_raw + (1.0 - self.alpha) * self.xdot_filt
```

3.5 Startup ramp. All error terms are scaled by a linear ramp over the first 2 seconds to prevent large torque spikes when the controller first activates:

```
ramp = min(time_now / self.ramp_time, 1.0)
```

3.6 Nullspace controller. The robot's redundant joints are used to hold a comfortable home posture without disturbing end-effector motion:

```

N_bar = np.eye(self.nv) - J.T @ J_bar.T
tau_null = N_bar @ (
    self.Kp_null @ (self.q0 - q) + self.Kd_null @ (-qd)
)

```

3.7 Task-space inertia. The dynamically consistent Λ and $\bar{J}$ are computed with damped least-squares regularization to prevent ill-conditioning near singularities:

```

def _compute_lambda_and_jbar(self, J, Mq):
    M_inv = np.linalg.inv(Mq)
    JMinvJt = J @ M_inv @ J.T
    Lambda = np.linalg.inv(JMinvJt + self.lambda_reg * np.eye(3))
    J_bar = M_inv @ J.T @ Lambda
    return Lambda, J_bar, M_inv

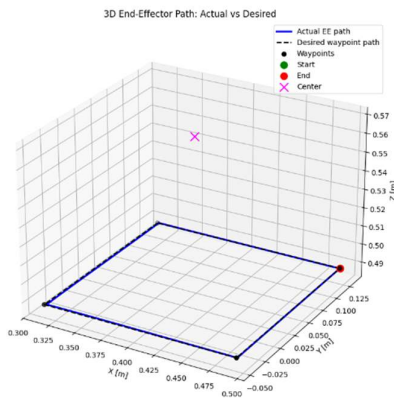
```

4. Simulation and Results:

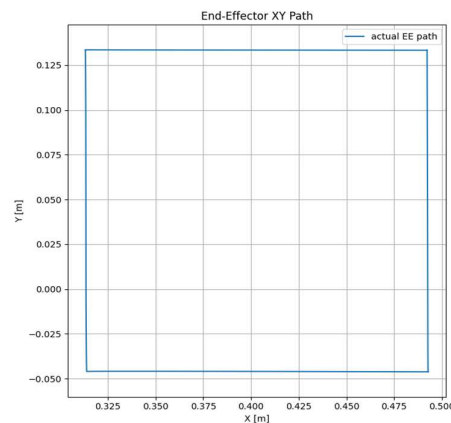
4.1 Normal Mode (No Bias Force): In normal mode, no external bias force is applied, so the UR5e operates under nominal conditions with only its internal dynamics and gravity acting on the manipulator. The cartesian impedance controller tracks the desired rectangular end-effector trajectory while exhibiting the tuned stiffness and damping behaviour, resulting in smooth motion and small steady-state tracking errors along all axes.

Trajectory Tracking:

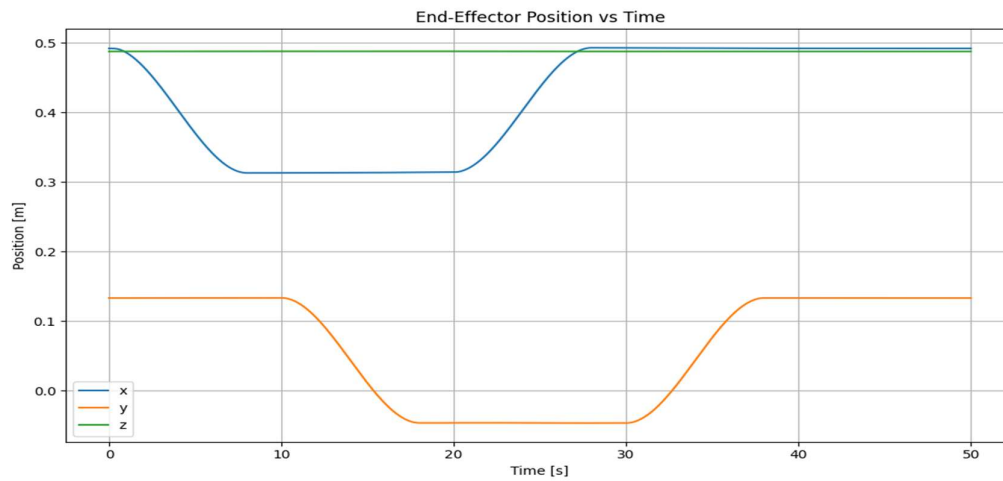
➤ 3D EE Path Actual vs Desired



➤ End-Effector XY Path

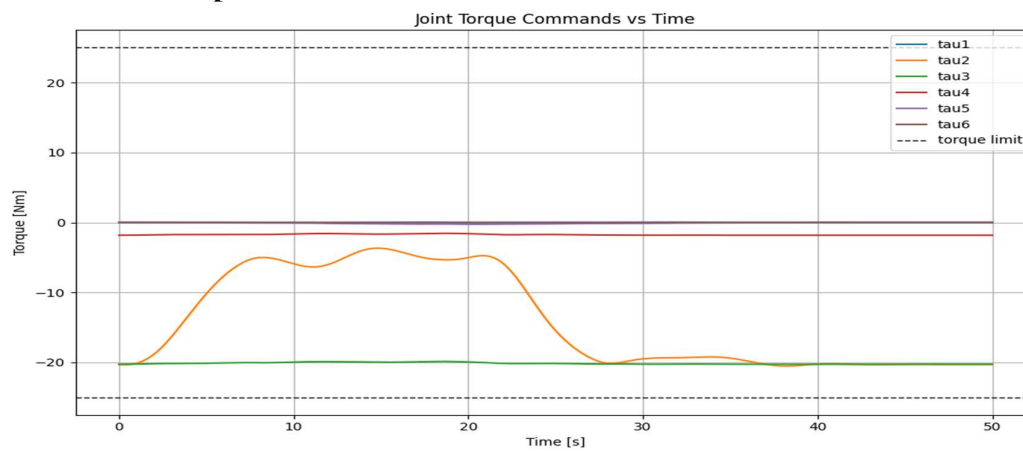


➤ End-effector Position Vs Time

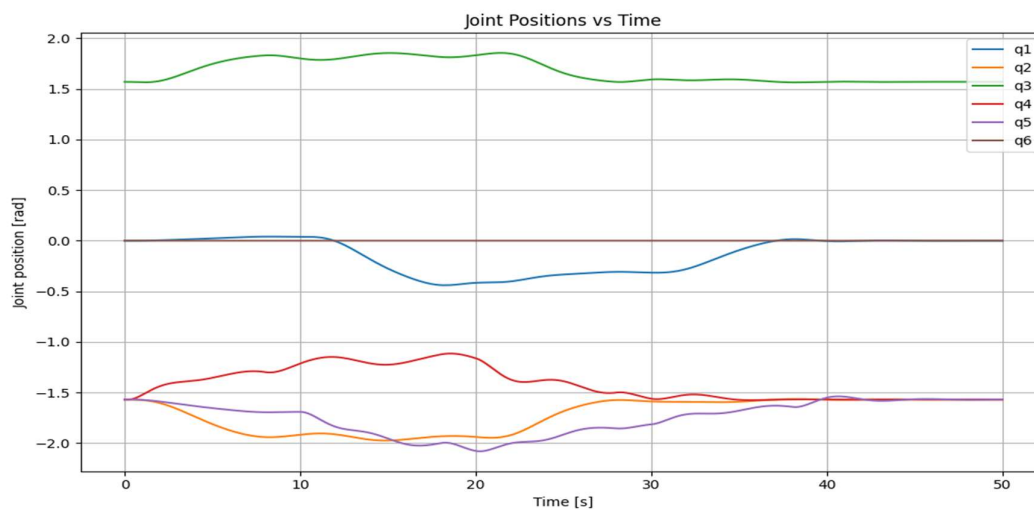


Joint Behaviour:

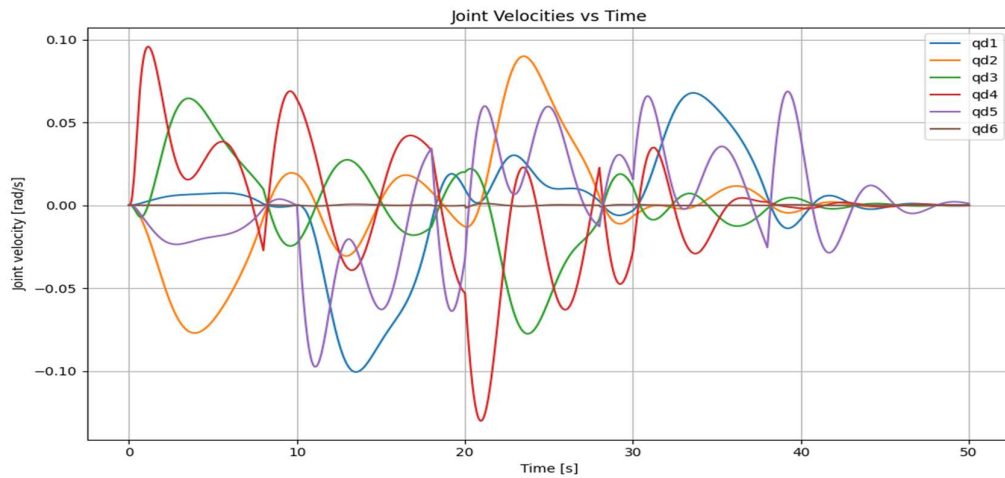
➤ Joint Torque Commands Vs Time



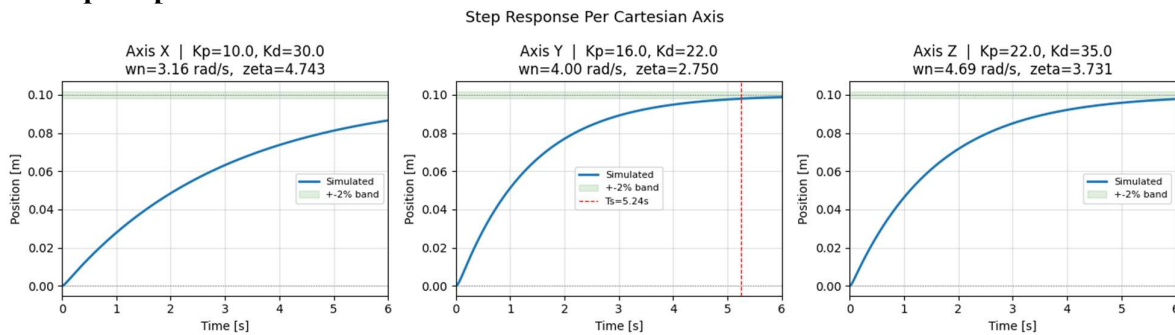
➤ Joint Position Vs Time



➤ Joint Velocities Vs Time



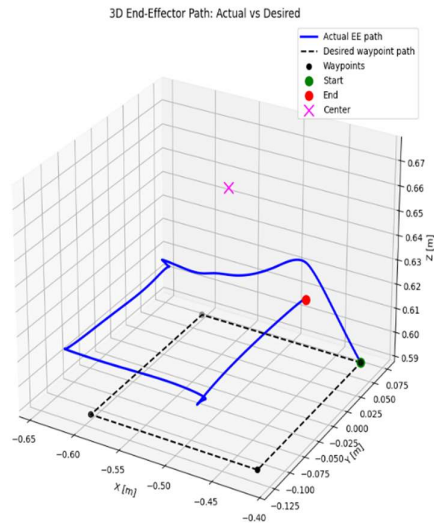
➤ Step Response Per Cartesian Axis



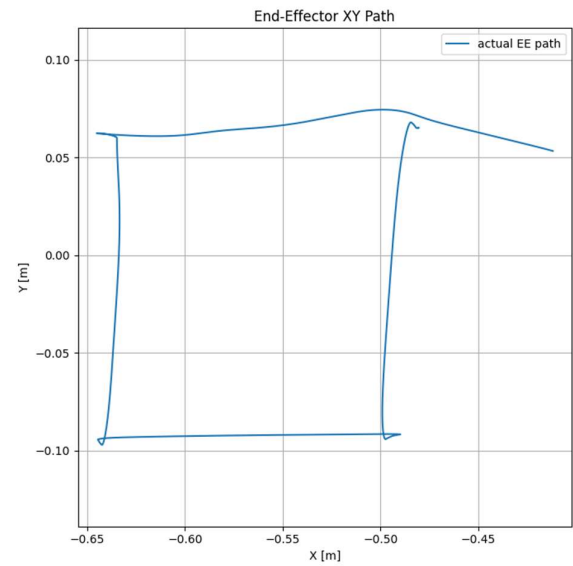
4.2 Hard Mode (Bias Force in X): In hard mode, an external bias force is applied in the xx -direction while the robot tracks the same Cartesian trajectory. The controller is designed so that this disturbance produces noticeable deflection primarily in xx , with minimal coupling into the yy and zz directions, demonstrating effective task-space decoupling and directional compliance. This test shows that the Cartesian impedance controller can reject or accommodate disturbances in a controlled way while preserving stable motion in the other axes.

Trajectory Tracking:

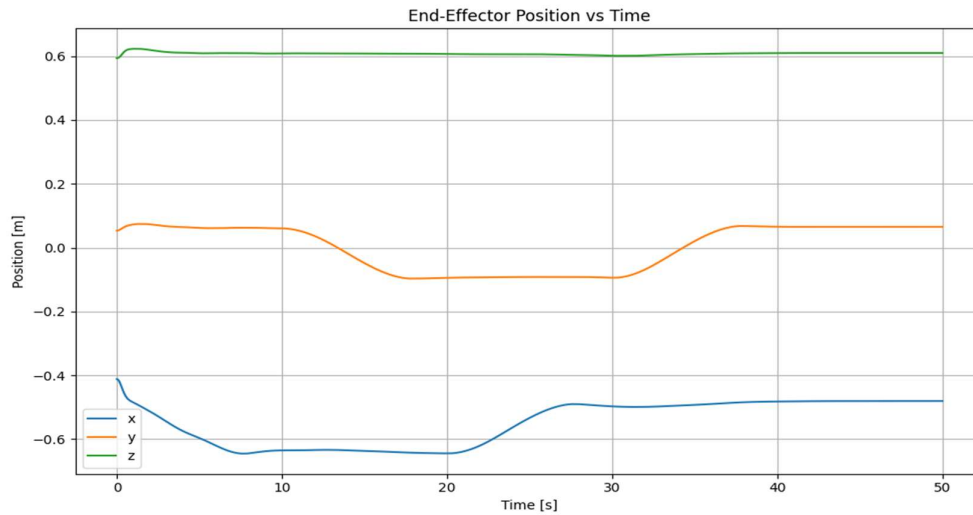
➤ 3D EE Path Actual vs Desired



➤ End-Effector XY Path

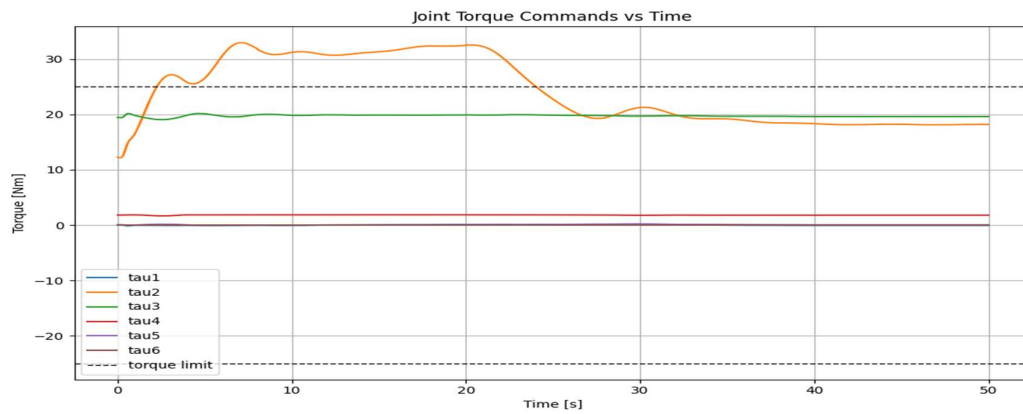


➤ End-effector Position Vs Time — X deflects, Y and Z unaffected

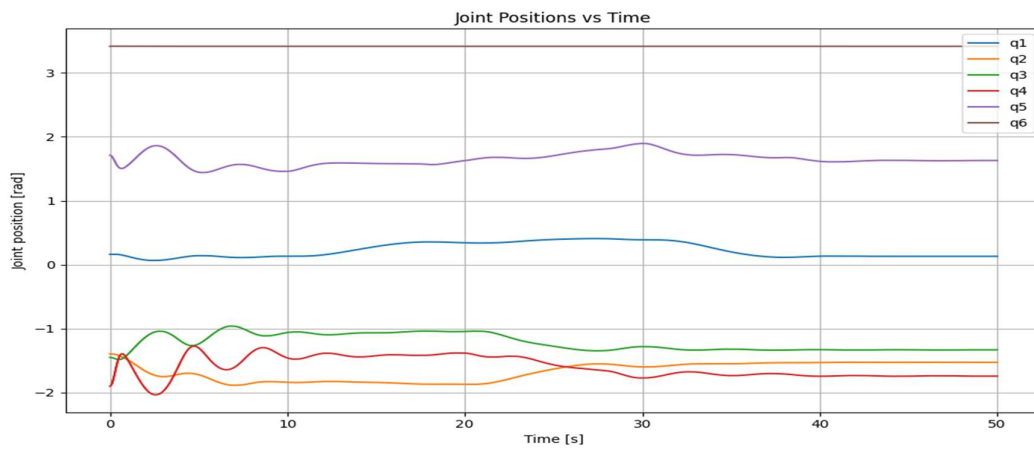


Joint Behaviour:

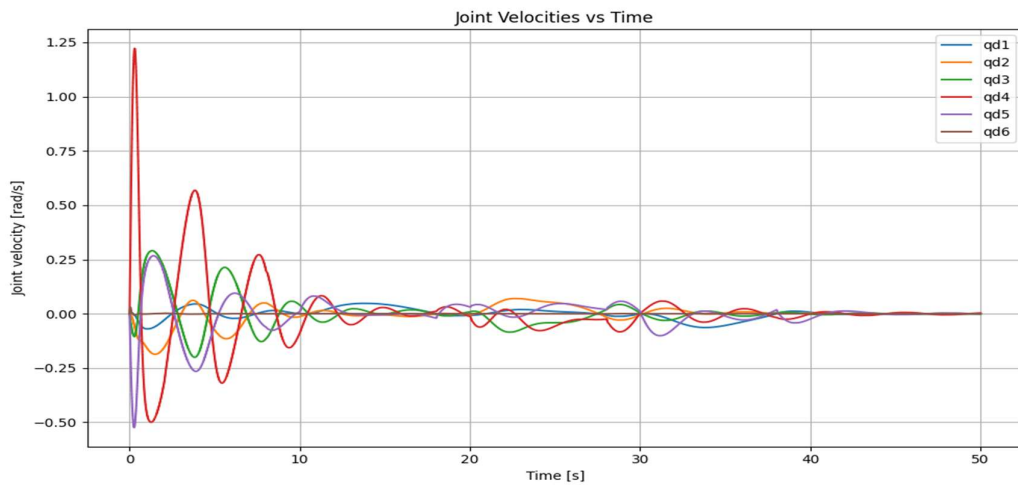
➤ Joint Torque Commands Vs Time



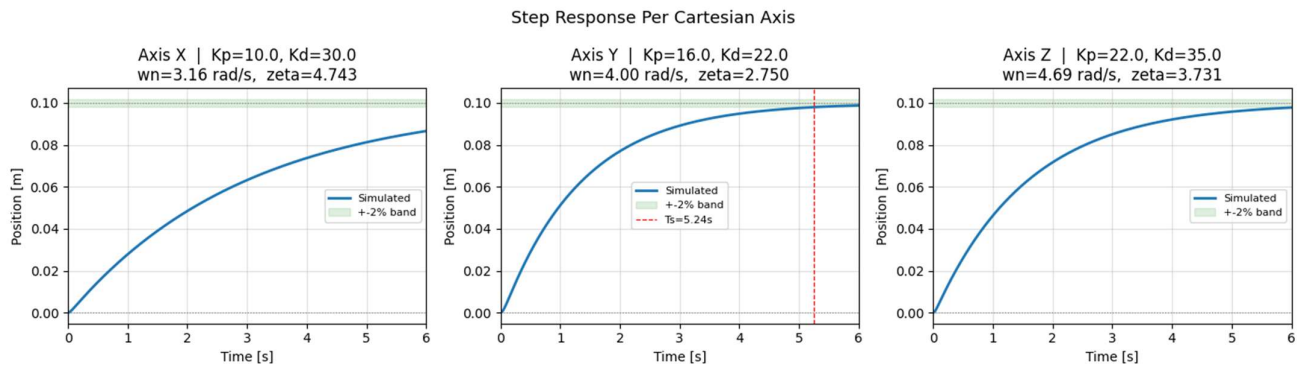
➤ Joint Position Vs Time



➤ Joint Velocities Vs Time



➤ Step Response Per Cartesian Axis



4.3 Discussion:

Metric	Normal Mode	Hard Mode (Bias in X)
X settling time	≈ 8 s	≈ 8 s
Y settling time	≈ 8 s	≈ 8 s
Z settling time	— (constant)	— (constant)
X overshoot	None	None
Y overshoot	None	None
Z overshoot	None	None
Damping ratio ζ	≈ 1	≈ 1
X deflection from bias force	—	Yes
Y deflection from bias force	—	No
Z deflection from bias force	—	No

Normal mode: The end-effector tracks the rectangular trajectory accurately with no overshoot in any axis, and Z remains constant at 0.49 m, showing the controller maintains height while producing a critically damped response with $\zeta \approx 1$. Joint torques and velocities stay within the UR5e limits and settle smoothly between waypoints, indicating stable and well-tuned impedance behaviour.

Hard mode: With the bias force applied, the trajectory deflects mainly in X while Y remains nearly unchanged, demonstrating axis decoupling in the Y direction. The distorted, shifted rectangle in the path

plots and the brief exceedance of the τ_2 torque limit show the controller working against the external load near its capacity, yet the system remains stable over the entire motion.

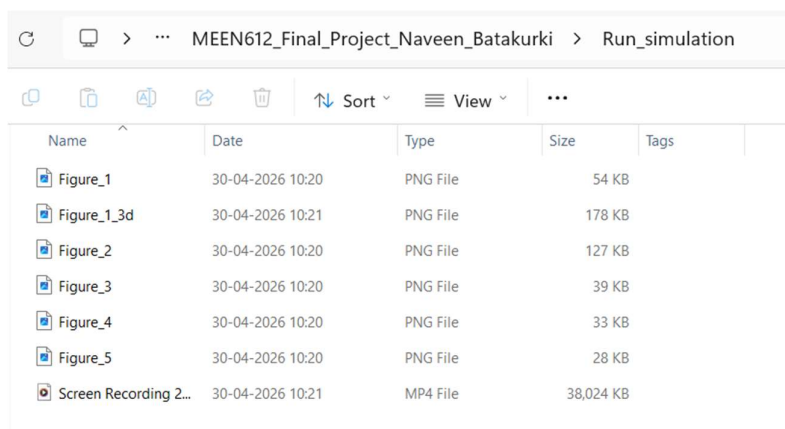
5. Conclusion:

This project demonstrated a Cartesian impedance controller for a UR5e that tracks a rectangular trajectory with near-critically damped behaviour and no overshoot on any axis in normal mode. Under a bias force in X, the controller produced deflection primarily along X while keeping Y largely decoupled, confirming effective task-space decoupling and stable operation within (or close to) hardware limits.

In Class demo: I have tried but robot simulation didn't work. But it was a good learning experience.

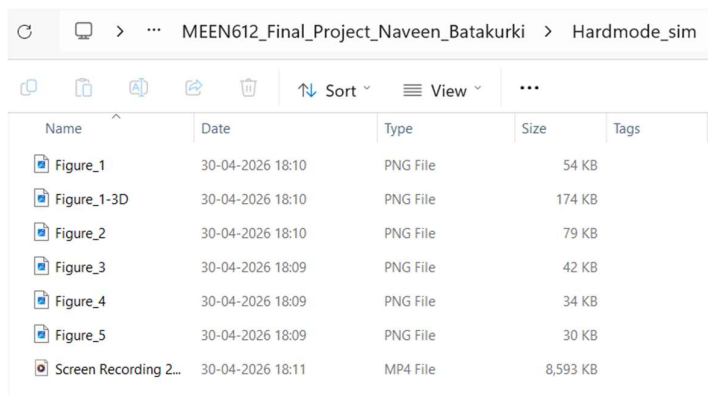
6. Document list:

Normal run: run_simulation.py



Name	Date	Type	Size	Tags
Figure_1	30-04-2026 10:20	PNG File	54 KB	
Figure_1_3d	30-04-2026 10:21	PNG File	178 KB	
Figure_2	30-04-2026 10:20	PNG File	127 KB	
Figure_3	30-04-2026 10:20	PNG File	39 KB	
Figure_4	30-04-2026 10:20	PNG File	33 KB	
Figure_5	30-04-2026 10:20	PNG File	28 KB	
Screen Recording 2...	30-04-2026 10:21	MP4 File	38,024 KB	

Hardmod run: Hardmod_sim.py



Name	Date	Type	Size	Tags
Figure_1	30-04-2026 18:10	PNG File	54 KB	
Figure_1-3D	30-04-2026 18:10	PNG File	174 KB	
Figure_2	30-04-2026 18:10	PNG File	79 KB	
Figure_3	30-04-2026 18:09	PNG File	42 KB	
Figure_4	30-04-2026 18:09	PNG File	34 KB	
Figure_5	30-04-2026 18:09	PNG File	30 KB	
Screen Recording 2...	30-04-2026 18:11	MP4 File	8,593 KB	

Code:

Sort

View

Name	Date modified	Type	Size
 hardmode_sim	30-04-2026 16:24	Python Source File	11 KB
 my_controller	30-04-2026 20:09	Python Source File	15 KB
 run_simulator	30-04-2026 20:28	Python Source File	10 KB